

MA-Métier : Autopilote

Documentation

Table des matières

Logique de base du vol plat :	3
Valeur à récupérer (Get).....	3
Valeur à définir (Set).....	3
Plat Roulis	4
Plat Tangage et Vitesse $N=vitesse\ demandée$	4
Direction $N=angle\ départ$	4
Problèmes.....	5
Problème 1 :	5
Problème 2 :	5
Boîte à outil commande SimConnect :	6
Description et objectif :	6
Exploration et expérimentation :	6
Testes :	6
Fonctionnement :	6
Correction théorique :	8
Ajout d'un mode de vol en cercle complet	9
Objectif général	9
Valeurs récupérées (Get)	9
Valeur définie (Set)	9
Déclenchement du cercle	9
Mémorisation du cap de départ.....	9
Logique du virage	10
Détection du cercle complet	10
Problème des angles 0 / 360	10
Solution : Normalisation des angles	10
Conditions de fin du cercle	11
Fin du cercle	11
Avantages de cette approche	11
Limites et améliorations possibles	11
Conclusion	12

Logique de base du vol plat :

Valeur à récupérer (Get)

- Boussole: MAGNETIC COMPAS
- Alt barométrique: PRESSURE ALTITUDE
- Vitesse: AIRSPEED INDICATED
- Angle Roulis: PLANE BANK DEGREES
- (Angle Tangage: PLANE PITCH DEGREES)

Valeur à définir (Set)

- Aileron: AILERON POSITION
- Profondeur: ELEVATOR POSITION
- Dérive: RUDDER POSITION
- (Aérofrein: SPOILERS HANDLE POSITION)

Plat Roulis

Objectif : Vol plat

Si Angle Roulis $> 10^\circ$

Set Aileron droite 5°

si Angle Roulis $< -10^\circ$

Set Aileron gauche 5°

Si Angle Roulis $< 10^\circ$ && Angle Roulis $> -10^\circ$

Set Aileron 0°

Plat Tangage et Vitesse N=vitesse demandée

Objectif : Vol à une vitesse donnée

Si Vitesse $> N+20$

Set Profondeur 5°

Si Vitesse $< N-20$

Set Profondeur -5°

Si Vitesse $< N+20$ && Vitesse $> N-20$

Set Profondeur 0°

Direction N=angle départ

Objectif : Toujours avoir la même direction.

Si Boussole $> N+10$

Set Dérive -5

Si Boussole $< N-10$

Set Dérive 5°

Si Boussole $< N+10$ && Boussole $> N-10$

Set Dérive 0°

Problèmes

Problème 1 :

Après teste il y a des conflits entre les différentes fonctionnalités comme « Direction » et « Plat Roulis ».

Problème 2 :

Plat Tangage et Vitesse » pose un problème.

Il s'agit d'une mauvaise gestion de l'inertie.

Si le planeur a une vitesse trop élevée, il va continuer à monter tant que la vitesse n'est pas $\leq N$. Ainsi, lorsque la vitesse devient $\leq N$, il est déjà trop tard : la correction n'est pas progressive, ce qui entraîne une perte excessive de vitesse pouvant aller jusqu'au décrochage. C'est l'effet « phugoïde ».

```
Connexion à Flight Simulator réussie
Appuie sur Q pour arrêter l'enregistrement
0140,733871535975 61,088951110839844 10,441435813903809 0,008637514400983478
0147,350071966019 59,21150207519531 7,176751136779785 0,005376054335568729
0105,724183635686 63,51224899291992 3,5449273586273193 0,003040312249935763
0008,95569910517 73,8079833984375 0,8529866337776184 0,003233842331236648
0848,817639858411 88,68428039550781 359,0058898925781 0,0037222936256909415
0648,674661507901 104,26130676269531 359,01959228515625 0,001689513866766887
0532,497429208302 112,47260284423828 4,254951000213623 0,001599251030264835
0542,5725197743 110,79302215576172 8,888252258300781 0,001105934510225354
0658,302266104192 100,4642562866211 12,567025184631348 0,0008278160918229274
0827,634971024827 83,77647399902344 15,277619361877441 0,0020865988717559446
0004,355110807051 62,35849380493164 17,714149475097656 0,00866190686096694
0133,644055483315 39,88492202758789 19,077579498291016 0,0062324118865479256
0199,88502833094 18,130434036254883 18,36408233642578 0,006768350963857855
0204,964907209036 0,47955411672592163 18,059967041015625 0,012379533881239912
0150,537748265599 0,00019309906929265708 14,324020385742188 0,011697479577848694
0058,217740819573 31,272748947143555 359,2813415527344 0,019345125857583058
0913,693889276929 56,14728546142578 285,8371276855469 -0,04397989694504576
```

Vous pouvez voir la vitesse à droite du trais bleu qui passe de plus de 100 noeux et quelque seconde après une vitesse de 0 (décrochage).

Solution possible :

Faire la gestion de la profondeur progressivement.

Faire la gestion de l'angle via l'angle de tangage et la vitesse.

Boîte à outil commande SimConnect :

Description et objectif :

La boîte à outil est une bibliothèque utilisable pour tous les programmes du projet. Elle doit contenir les fonctionnalités nécessaires pour piloter le planeur sur FlightSimulator. Il ne s'agit pas d'un exécutable, mais vraiment d'une pièce à intégrer à chaque projet qui requiert de piloter l'appareil.

Exploration et expérimentation :

L'expérimentation pour réaliser la boîte à outil est chaotique. Les documentations sur SimConnect se contredisent, les exemples provoquent des erreurs sans même pouvoir se compiler, et même les mécanismes de SimConnect demandent parfois un dll obsolète ou un package plus ou moins utilisable. Il aura fallu des erreurs créer une première version qui ne se lancera pas malgré une longue session pour régler les soucis de compatibilités.

La boîte à outil étant nécessaire pour l'équipe pour travailler sur du concret, une révision d'efficacité a été entreprise et a permis de parvenir à une version 2 fonctionnelle bien que moins maîtrisée.

Testes :

Afin d'assurer le fonctionnement de la boîte à outil, deux exécutables ont été créés. Le premier consistait à lancer une liste d'instruction de pilotage, une après l'autre, à chaque pression d'une touche de clavier. Le second était un test interactif, les instructions étaient brutes, mais pouvaient être choisies sur le clavier à plusieurs reprises.

Le résultat a été positif même si le planeur ne restait pas longtemps en vol.

Fonctionnement :

Toutes les commandes commencent par un await, son objectif est d'attendre que le simulateur ait bien reçu l'instruction avant de continuer à l'exécuter. Ceci permet entre autres d'assurer une coordination entre la commande demandée par le programme et sa réalisation sur le simulateur.

Ces commandes sont construites par le constructeur présent dans commandes.cs suivant ce procédé :

```
public SimConnectControls(SimConnectClient sim)
{
    _sim = sim ?? throw new ArgumentNullException(nameof(sim));
}
```

Bouger l'aileron :

```
await controls.SetAileron("valeur");
```

La valeur attendue par la commande permet de déterminer quel aileron doit faire quoi. Elle attend une valeur comprise entre -1 et +1. Par exemple, une valeur à -1 va demander à l'aileron gauche de s'incliner au maximum, à l'inverse, +1 va plutôt demander à l'aileron droit. La valeur 0 consiste à demander aux ailerons de ne plus avoir d'inclinaison.

Commande construite :

```
public async Task SetAileron(double value)
{
    value = Math.Max(-1.0, Math.Min(1.0, value));
    await _sim.SimVars.SetAsync("AILERON POSITION", "position", value);
}
```

Changer l'elevateur :

```
await controls.SetElevator("valeur");
```

L'elevateur fonctionne de la même manière, il détermine l'inclinaison du nez de l'appareil, avec un +1 nous demandons au nez de se lever au maximum vers le haut tandis avec -1 de descendre, ce qui correspond à une piquée dans ce dernier cas. La valeur 0 est donc pour rester plat.

Commande construite :

```
public async Task SetElevator(double value)
{
    value = Math.Max(-1.0, Math.Min(1.0, value));
    await _sim.SimVars.SetAsync("ELEVATOR POSITION", "position", value);
}
```

Changer le rudder :

```
await controls.SetRudder("valeur");
```

Le rudder est similaire aux ailerons, c'est un élément déterminant pour un dérapage ou pour mener un virage efficacement.

Commande construite :

```
public async Task SetElevator(double value)
{
    value = Math.Max(-1.0, Math.Min(1.0, value));
    await _sim.SimVars.SetAsync("ELEVATOR POSITION", "position", value);
}
```

Correction théorique :

Initialement, il était pensé de créer les commandes dans un fichier et de les référencer dans un fichier interface afin qu'elles puissent être appelées dans l'exécutable. Cette théorie était erronée, l'interface n'étant en réalité pas nécessaire bien qu'elle soit un bon élément.

Ajout d'un mode de vol en cercle complet

Objectif général

L'objectif est de permettre à l'avion de réaliser un cercle complet automatique vers la gauche, tout en conservant la logique de base du vol plat existant. Le cercle est déclenché manuellement et se termine automatiquement lorsque l'avion revient à sa direction de départ.

Valeurs récupérées (Get)

- Boussole : MAGNETIC COMPASS Permet de connaître le cap actuel de l'avion
- Angle de roulis : PLANE BANK DEGREES Utilisé pour vérifier l'inclinaison de l'avion pendant le virage
- Vitesse : AIRSPEED INDICATED Influence indirectement le rayon du cercle

Valeur définie (Set)

- Ailerons : AILERON POSITION Utilisés pour incliner l'avion vers la gauche
- Dérive : RUDDER POSITION Utilisé pour maintenir un virage stable
- Profondeur : ELEVATOR POSITION Utilisés pour faire le virage correctement

Déclenchement du cercle

Le cercle est déclenché par l'utilisateur à l'aide du clavier :

- Appui sur la touche C
- Activation d'une variable d'état doCircle Tant que cette variable est active, le mode vol en ligne droite est désactivé.

Mémorisation du cap de départ

Au début du cercle, le cap est stocké dans une variable circleStartHeading, et une variable leftStartHeading est initialisée à false. Ces variables permettent de savoir quand l'avion a quitté le cap de départ, et quand il y est revenu après un tour complet.

Logique du virage

Pendant le cercle :

- Les ailerons sont fixés à une valeur constante vers la gauche
- La dérive est également appliquée vers la gauche
- La profondeur est également appliquée vers le haut

Cela permet d'obtenir :

- Un virage fluide
- Un cercle continu

Le rayon du virage dépend de :

- La vitesse de l'avion
- L'angle appliqué aux commandes

Détection du cercle complet

Problème des angles 0 / 360

Les angles de cap étant compris entre 0 et 360, une comparaison directe peut provoquer des erreurs lors du passage par 0.

Exemple :

Cap de départ : 350

Cap actuel : 10

$10 - 350 = -340$ On obtient une valeur très grande, alors qu'en réalité l'avion ne s'est écarté que de 20 degrés. Ce type de calcul fausse la détection du retour au cap de départ et de la fin du cercle.

Solution : Normalisation des angles

La fonction **NormalizeAngle** est utilisée pour ramener toute différence de cap dans l'intervalle -180, 180

Cela permet une comparaison fiable du cap, quelle que soit la position de l'avion.

Conditions de fin du cercle

Le cercle est considéré comme terminé lorsque :

- L'avion s'est éloigné du cap de départ de plus de 80°
- L'avion revient à moins de 80 ° de ce même cap

Ces deux conditions évitent un arrêt immédiat du cercle au moment du déclenchement.

Fin du cercle

Lorsque le cercle est terminé :

- Les ailerons et la dérive sont remis à 0
- La variable **doCircle** est désactivée
- Les variables du cercle sont réinitialisées
- L'avion repasse automatiquement en vol en ligne droite

Avantages de cette approche

- Le cercle est basé sur la direction, pas sur le temps
- La logique est robuste face aux variations de vitesse
- Aucun conflit avec le vol plat existant
- Le système est facilement extensible :
 - o Cercle à droite
 - o Enchaînement de manœuvres

Limites et améliorations possibles

- Le rayon du cercle n'est pas constant
- L'altitude peut légèrement varier
- Une gestion plus fine du roulis pourrait améliorer la stabilité
- Possibilité d'ajouter une correction progressive via le roulis

Conclusion

Cette implémentation permet de réaliser un cercle complet de manière fiable et contrôlée, tout en conservant le fonctionnement du vol en ligne droite. La séparation des modes de vol évite les conflits et facilite la maintenance du code.